
GUIDED FLOW MATCHING
FOR
EXTREME WEATHER SIMULATION

MASTER'S THESIS

AUTHOR

ALAIN JOSS

ALAIN.JOSS@UZH.CH

SUPERVISORS

MAXIM SAMARIN

SHIRIN GOSHTASBPOUR

PROF. DR. GUILLAUME OBOZINSKI

SWISS DATA SCIENCE CENTER

DATE OF SUBMISSION: AUGUST 7, 2026

Abstract

Contents

1	Introduction	1
2	Background	4
2.1	Flow matching	4
2.1.1	Conditional probability paths	5
2.1.2	Conditional and marginal velocity fields	6
2.1.3	Flow-matching objective	6
2.1.4	Linear probability path	7
2.2	Guidance	8
2.2.1	Guidance as posterior sampling	8
2.2.2	From scores to flow-matching velocities	9
2.2.3	Guidance-strength schedules	10
2.3	ArchesWeather	11
2.3.1	Dataset	11
2.3.2	Forecasting setup	12
2.3.3	Residual flow model	13
2.3.4	Sampling	14
3	Background	16
3.1	Flow matching	16
3.2	Guidance	17
3.2.1	Guidance as posterior sampling	18
3.2.2	From diffusion to flow matching	18
3.2.3	Guidance strength-schedule	19

3.2.4	Guidance methods	20
3.3	ArchesWeather	21
3.3.1	Dataset	21
3.3.2	Objective	21
3.3.3	Sampling	23
3.4	Guidance in weather modeling	24
4	Specifying events of interest	25
4.1	Target variable and region: mask	25
4.2	Target trajectory	26
4.3	Pipeline overview	27
5	Guidance design	28
5.1	Base guidance method	28
5.1.1	Adapting DPS	28
5.1.2	Guidance strength-schedule	30
5.1.3	Base algorithm	30
5.2	Guidance design choices	31
5.2.1	Guidance schedule	31
5.2.2	Guidance loss	31
5.3	Specific guidance methods (instances)	32
5.3.1	CLOSE-GAP	33
5.3.2	OPTIMIZE-SCHEDULE	33
5.3.3	OPTIMIZE-GAIN	34
5.3.4	Comparison	34
6	Evaluation	36
6.1	Flow-time assessment	37
6.1.1	Tests	37
6.2	Weather-time assessment	38
6.2.1	Parameter characterisation tests	38
6.2.2	Realism tests	39

6.3	Method Validation	40
7	Experiments	41
7.1	Flow-time evaluation	41
7.1.1	Setup	41
7.1.2	Achievement tests	41
7.2	Weather-time evaluation	42
7.2.1	Parameter characterisation tests	42
7.2.2	Realism tests	42
7.3	Use cases	42
7.3.1	Europe 2020-06-01	43
7.3.2	Australia 2020-01-01	43
7.3.3	North America 2020-07-01	43
8	Discussion	44
8.1	Mask	44
8.2		45
9	Conclusion	46
9.1	Summary	46
9.2	Future Directions	46
9.3	Conclusive Remarks	46
Appendices		
A	Code-base structure	48

List of Figures

2.1	The two nested time scales in ArchesWeather. Each autoregressive weather step $n \rightarrow n + 1$ contains a complete flow of $T = 25$ steps used to generate the forecast residual.	15
2.2	Computational graph of one ArchesWeather forecast step. The deterministic model produces the base forecast $\hat{x}_{n+1}^{\text{det}}$, while the flow model generates the residual through T Euler steps before both components are combined.	15
3.1	The two nested time scales: the weather time n advances in 24 h steps, while a full flow of T steps generates each transition.	22
3.2	One forecast step of ArchesWeather.	23

List of Tables

Chapter 1

Introduction

Machine learning models have reached state-of-the-art skill in weather forecasting, where they now rival traditional physics-based solvers at a fraction of their computational cost. One class among them are generative probabilistic models. First developed in the image domain, they have recently been carried over to weather. There, they now define the state of the art in probabilistic forecasting.

In the image domain, the applications of interest almost all *condition* a pretrained model rather than sample from it unconditionally: text-to-image generation, editing, and inpainting each steer the generative process towards a desired target through *guidance*. In the weather domain, by contrast, generative models are mostly used in their unguided form, as residual predictors that both overcome the oversmoothing of deterministic models and enable ensemble forecasts out of the box. The use of guidance in the weather domain remains largely unexplored. The few exceptions steer forecasts towards specific extreme events: [cite] generate windstorms, [cite] minimise precipitation, and [cite] produce heatwaves.

— frame it in the style of the presentation → state what we want like in the presentation and also state the high level objective slide contents — we could also take inspiration from the proposal Extreme events cause a disproportionate share of weather-related damage, yet their rarity makes them precisely the cases in which forecast ensembles are thinnest and observational data for study scarcest. This motivates our central question: can we deliberately generate realistic extreme weather trajectories with a generative forecasting model, rather than wait for an unguided ensemble to sample them? Building on the recent works above, we tackle the question by splitting it into three subproblems—specifying the event, guiding the model, and evaluating the result—which together form a general framework to *specify*, *generate*, and *evaluate* extreme events with generative weather models.

But why do we need to adapt guidance methods from the image domain to the weather domain in the first place? The answer lies in three key differences between the two domains. — insert here the differences from the presentation

We build on ArchesWeather, a residual flow-matching model, and focus throughout on surface-temperature heatwaves over a chosen region of interest; the methodology itself is model and target agnostic. The three subproblems are also our three contributions:

1. **Specify.** A heuristic that uses historical states consistent with a given start state to define a plausible target trajectory for the region and variable of interest.
2. **Guide.** A gradient-based guidance method, adapted from classifier guidance, that steers the forecast onto the specified target by driving the gap to zero rather than by tuning guidance strength for a visually plausible result, thus requiring minimal tuning and minimising the intervention.
3. **Evaluate.** A set of flow and weather plausibility tests. In particular, for output plausibility we propose a benchmark, built from the same historical data, that tests generated trajectories for spatial, pressure-level, and cross-variable coherence, using the historical states as anchors and the unguided forecast as baseline.

The remainder of the thesis is organised as follows. Chapter 2 introduces the technical background on flow matching, guidance, generative weather models, and seminal applications of guidance in the weather domain. Chapters 4 to 6 develop the three phases of the framework in turn: specifying the target event, designing the guidance, and defining the evaluation. Chapter 7 presents the experiments and Chapter 8 discusses them. Chapter 9 concludes with a summary, directions for future work, and final remarks.

— state previously that our event of interest are heatwaves

— events in the future can be much more extreme than in the past -> see tsunami Fukushima, wildfires, ... Everything is unfolding in front of our eyes I'm writing this paper. We are thus interested in exploiting the capability of diffusion models to guide towards more extreme events of the past. We want to exploit the learned transition function to make the guided trajectory a coherent sequence of weather states, across time, pressure levels, spatially, and of course across variables (realistic and representative covariation). In other words, we want to generate an extreme event, as we would have measured it from ground truth data.

— explain somewhere that everything runs on two clocks.

Here we might have in mind multiple ways to use this

If our method is successful, these are two usage modalities that come to mind w.r.t. to extreme

temperatures:

- Define extreme trajectories from the current state into the future. TODO: include presentation reasoning about mapping bottlenecks
- Take past heatwaves and guide them towards even higher extremes.

For both use cases we might sample multiple trajectories at multiple intensity levels to guide towards.

— Specify that guidance looks promising particularly in the context of extreme events, but is not specific to them. It could be used to generate any alternative weather scenario that diverges from the base forecast.

Chapter 2

Background

This chapter introduces the theoretical and methodological background required for the development of our guidance approach.

We begin with flow matching, the generative framework underlying ArchesWeather, and introduce the notion of probability paths, noisy states, velocity fields, and clean estimates used throughout this work. We then formulate guidance from a posterior-sampling perspective and derive how conditioning modifies the flow-matching velocity field.

Next, we describe the components of ArchesWeather relevant to our method, including its deterministic forecast, residual flow model, and sampling procedure. Finally, we review existing work on guidance for generative weather models and position our approach within the current literature.

2.1 Flow matching

Generative modeling frames the task of generating objects $x_1 \in \mathbb{R}^d$ as sampling from a data distribution p_1 . Flow matching [4] models p_1 as the endpoint of a continuous probability path

$$(p_t)_{t \in [0,1]},$$

which transports a tractable source distribution p_0 , typically a standard Gaussian, to the data distribution p_1 .

At flow time t , we denote the current state along this path by z_t , with

$$z_0 \sim p_0, \quad z_t \sim p_t, \quad z_1 = x_1 \sim p_1.$$

Throughout this work, we refer to z_0 as the *noise*, intermediate states z_t , $t \in (0, 1)$, as *noisy states*, and the endpoint $z_1 = x_1$ as the *clean state*.

The evolution of z_t is governed by a time-dependent velocity field u_t , defining the ordinary differential equation

$$\frac{d}{dt}z_t = u_t(z_t). \quad (2.1)$$

Starting from $z_0 \sim p_0$ and integrating this ODE from $t = 0$ to $t = 1$ therefore transforms samples from the source distribution into samples from the data distribution.

2.1.1 Conditional probability paths

Directly specifying the marginal probability path p_t and its corresponding velocity field u_t is generally intractable. Flow matching circumvents this problem by defining a tractable *conditional probability path*

$$p_t(\cdot \mid x_1)$$

for each data sample $x_1 \sim p_1$.

A common choice is the Gaussian conditional probability path

$$p_t(\cdot \mid x_1) = \mathcal{N}(\alpha_t x_1, \beta_t^2 I), \quad (2.2)$$

where the schedules $\alpha_t, \beta_t \in [0, 1]$ interpolate between noise and data according to

$$\alpha_0 = 0, \quad \beta_0 = 1, \quad \alpha_1 = 1, \quad \beta_1 = 0.$$

Consequently,

$$p_0(\cdot \mid x_1) = \mathcal{N}(0, I), \quad (2.3)$$

$$p_1(\cdot \mid x_1) = \delta_{x_1}. \quad (2.4)$$

Sampling from the conditional path is equivalent to interpolating a data sample $x_1 \sim p_1$ with independent Gaussian noise $\varepsilon \sim \mathcal{N}(0, I)$:

$$z_t = \alpha_t x_1 + \beta_t \varepsilon \sim p_t(\cdot \mid x_1). \quad (2.5)$$

Marginalizing over the data distribution gives the corresponding marginal probability path

$$p_t(z_t) = \int p_t(z_t \mid x_1) p_1(x_1) dx_1. \quad (2.6)$$

2.1.2 Conditional and marginal velocity fields

Each conditional probability path $p_t(\cdot \mid x_1)$ is generated by a corresponding conditional velocity field $u_t(z_t \mid x_1)$.

For the Gaussian path in Equation (2.2), differentiating the interpolation in Equation (2.5) gives

$$u_t(z_t \mid x_1) = \dot{\alpha}_t x_1 + \dot{\beta}_t \varepsilon. \quad (2.7)$$

Substituting

$$\varepsilon = \frac{z_t - \alpha_t x_1}{\beta_t}$$

yields

$$u_t(z_t \mid x_1) = \frac{\dot{\beta}_t}{\beta_t} z_t + \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) x_1. \quad (2.8)$$

The marginal velocity field generating p_t can then be expressed as the conditional velocity averaged over all clean samples x_1 compatible with the current noisy state z_t :

$$u_t(z_t) = \mathbb{E}_{x_1 \sim p_t(\cdot \mid z_t)} [u_t(z_t \mid x_1)]. \quad (2.9)$$

While this marginal velocity is generally not available in closed form, the conditional velocity in Equation (2.30) is tractable during training, since both x_1 and the sampled noise ε are known.

2.1.3 Flow-matching objective

Flow matching exploits the fact that a neural network can be trained on the tractable conditional velocity field while learning the corresponding marginal velocity field [4]. Writing $u_\theta(z_t, t)$ for the learned velocity field, the conditional flow-matching objective is

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}[0,1], \\ x_1 \sim p_1, \\ z_t \sim p_t(\cdot \mid x_1)}} [\|u_\theta(z_t, t) - u_t(z_t \mid x_1)\|_2^2]. \quad (2.10)$$

This conditional objective has the same gradient with respect to the model parameters as the corresponding, but intractable, marginal flow-matching objective [4].

After training, $u_\theta(z_t, t)$ approximates the marginal velocity field $u_t(z_t)$. Generation then consists of sampling

$$z_0 \sim p_0$$

and numerically integrating

$$\frac{d}{dt}z_t = u_\theta(z_t, t) \quad (2.11)$$

from $t = 0$ to $t = 1$, such that the resulting endpoint z_1 approximately follows p_1 .

2.1.4 Linear probability path

A particularly simple choice is the linear, or optimal-transport, probability path

$$z_t = tx_1 + (1 - t)\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I), \quad (2.12)$$

corresponding to

$$\alpha_t = t, \quad \beta_t = 1 - t.$$

Its conditional velocity reduces to the constant displacement

$$u_t(z_t \mid x_1) = x_1 - \varepsilon. \quad (2.13)$$

Training therefore amounts to least-squares regression onto the displacement between the source noise and the clean sample.

At an intermediate flow time $t < 1$, the noisy state z_t does not uniquely identify the clean sample x_1 from which it originates. Instead, the possible clean samples are described by the conditional distribution

$$p_t(x_1 \mid z_t).$$

In practice, a generative model can provide a tractable clean estimate

$$\hat{x}_t := \hat{x}_\theta(z_t, t) \quad (2.14)$$

from the current noisy state. The distinction between the full conditional distribution $p_t(x_1 \mid z_t)$ and the point estimate $\hat{x}_t$ will become important when introducing guidance in the following section.

2.2 Guidance

In unconditional generation, the flow-matching model produces samples from the data distribution $p_1(x_1)$. In many applications, however, we are interested in generating samples that satisfy a desired condition y . Guidance modifies the sampling process to favor samples from the corresponding conditional distribution

$$p_1(x_1 \mid y).$$

We first formulate guidance from a posterior-sampling perspective and then translate the resulting score modification to the flow-matching velocity field. Finally, we introduce a general guidance-strength schedule that will be used throughout this work.

2.2.1 Guidance as posterior sampling

Rather than considering only the conditional distribution at the clean endpoint, we can define the corresponding distribution over noisy states at any flow time t . By Bayes' rule,

$$p_t(z_t \mid y) \propto p_t(z_t) p_t(y \mid z_t). \quad (2.15)$$

Taking the gradient with respect to the noisy state gives the conditional score

$$\nabla_{z_t} \log p_t(z_t \mid y) = \nabla_{z_t} \log p_t(z_t) + \nabla_{z_t} \log p_t(y \mid z_t). \quad (2.16)$$

The first term corresponds to the unconditional generative process, while the second introduces the desired condition y . Guidance can therefore be interpreted as modifying the unconditional generation process through the likelihood gradient

$$\nabla_{z_t} \log p_t(y \mid z_t).$$

In general, however, the intermediate likelihood $p_t(y \mid z_t)$ is not directly available. We therefore marginalize over the clean sample x_1 and assume that the condition y depends on the

generative process only through x_1 :

$$p_t(y \mid z_t) = \int p_t(y \mid x_1, z_t) p_t(x_1 \mid z_t) dx_1 \quad (2.17)$$

$$= \int p(y \mid x_1) p_t(x_1 \mid z_t) dx_1, \quad y \perp z_t \mid x_1 \quad (2.18)$$

$$= \mathbb{E}_{x_1 \sim p_t(\cdot \mid z_t)} [p(y \mid x_1)] \quad (2.19)$$

$$\approx \int p(y \mid x_1) \delta_{\hat{x}_t}(x_1) dx_1, \quad p_t(x_1 \mid z_t) \approx \delta_{\hat{x}_t}(x_1) \quad (2.20)$$

$$= p(y \mid \hat{x}_t). \quad (2.21)$$

Here, $\hat{x}_t = \hat{x}_\theta(z_t, t)$ denotes the clean estimate obtained from the current noisy state. The approximation replaces the full conditional distribution $p_t(x_1 \mid z_t)$ by a point mass at $\hat{x}_t$, yielding

$$p_t(y \mid z_t) \approx p(y \mid \hat{x}_t). \quad (2.22)$$

The likelihood $p(y \mid x_1)$ can in principle be modeled in different ways. A convenient choice for gradient-based guidance is to define it through a differentiable loss $\mathcal{L}$:

$$p(y \mid x_1) \propto \exp(-\mathcal{L}(x_1, y)). \quad (2.23)$$

It follows that

$$\nabla_{z_t} \log p(y \mid \hat{x}_t) = -\nabla_{z_t} \mathcal{L}(\hat{x}_t, y), \quad (2.24)$$

where the gradient is propagated through the clean estimate $\hat{x}_t = \hat{x}_\theta(z_t, t)$.

2.2.2 From scores to flow-matching velocities

The previous derivation expresses guidance in terms of scores. To apply it to flow matching, we require the corresponding modification of the velocity field.

For the Gaussian probability path introduced in Equation (2.2), the marginal velocity field and score are related by

$$u_t(z_t) = a_t \nabla_{z_t} \log p_t(z_t) + b_t z_t, \quad (2.25)$$

where

$$a_t := \beta_t^2 \left(\frac{\dot{\alpha}_t}{\alpha_t} - \frac{\dot{\beta}_t}{\beta_t} \right), \quad (2.26)$$

$$b_t := \frac{\dot{\alpha}_t}{\alpha_t}. \quad (2.27)$$

Applying the same relation to the conditional distribution gives

$$u_t(z_t \mid y) = a_t \nabla_{z_t} \log p_t(z_t \mid y) + b_t z_t. \quad (2.28)$$

Substituting the conditional-score decomposition yields

$$u_t(z_t \mid y) = a_t [\nabla_{z_t} \log p_t(z_t) + \nabla_{z_t} \log p_t(y \mid z_t)] + b_t z_t \quad (2.29)$$

$$= u_t(z_t) + a_t \nabla_{z_t} \log p_t(y \mid z_t). \quad (2.30)$$

Using the clean-estimate approximation derived above, we obtain

$$u_t(z_t \mid y) \approx u_t(z_t) + a_t \nabla_{z_t} \log p(y \mid \hat{x}_t). \quad (2.31)$$

Under the loss-based likelihood in Equation (2.23), this can equivalently be written as

$$u_t(z_t \mid y) \approx u_t(z_t) - a_t \nabla_{z_t} \mathcal{L}(\hat{x}_t, y). \quad (2.32)$$

Thus, gradient-based guidance in flow matching consists of evaluating a loss on the clean estimate $\hat{x}_t$, differentiating it with respect to the current noisy state z_t , and using the resulting gradient to modify the velocity field.

2.2.3 Guidance-strength schedules

The coefficient a_t in Equation (2.32) is determined by the underlying probability path. Using this coefficient preserves the posterior-sampling interpretation, up to the clean-estimate approximation introduced above.

In practice, guidance methods often rescale this contribution to obtain stronger or weaker control over the generated samples. We therefore replace a_t by a general guidance-strength schedule

$$\lambda_t := w \gamma_t c_t, \quad (2.33)$$

where

- $w \geq 0$ controls the overall guidance strength;
- $\gamma_t \geq 0$ controls how guidance varies along the flow;
- $c_t > 0$ is an optional algorithm-specific normalization factor.

We then define the *guided velocity field* as

$$\tilde{u}_t(z_t | y) := u_t(z_t) + \lambda_t \nabla_{z_t} \log p(y | \hat{x}_t) \quad (2.34)$$

$$= u_t(z_t) - \lambda_t \nabla_{z_t} \mathcal{L}(\hat{x}_t, y). \quad (2.35)$$

We refer to $u_t(z_t)$ as the *unguided velocity field*.

Setting

$$\lambda_t = a_t$$

recovers the approximate conditional velocity field in Equation (2.31). For a general choice of λ_t , however, the resulting dynamics no longer necessarily correspond to sampling from $p_1(x_1 | y)$. Instead, the guidance strength becomes an algorithmic parameter controlling the trade-off between the original generative dynamics and the imposed condition.

2.3 ArchesWeather

ArchesWeather [2] is a probabilistic medium-range weather forecasting model that combines a deterministic forecast with a flow-matching model of the forecast residual. In this section, we describe the components required for the development of our guidance method and refer to the original work for architectural and training details.

2.3.1 Dataset

ArchesWeather is trained on the ERA5 reanalysis dataset [3], using one of the versions preprocessed by WeatherBench 2 [7]. The data are represented on a regular 1.5° latitude–longitude grid with 121×240 spatial points.

Each weather state

$$x_n \in \mathbb{R}^{82 \times 121 \times 240}$$

contains 82 atmospheric fields. Four are surface variables: 2 m temperature, mean sea-level pressure, and the two components (u and v) of the 10 m wind. The remaining 78 fields

correspond to six upper-air variables

$$\{\text{geopotential, temperature, specific humidity, } u, v, \text{ vertical velocity}\}$$

evaluated at the 13 pressure levels

$$\{50, 100, 150, 200, 250, 300, 400, 500, 600, 700, 850, 925, 1000\} \text{ hPa.}$$

Weather states are available at 00:00, 06:00, 12:00, and 18:00 UTC. Following the Weather-Bench 2 evaluation protocol, dates from January 2020 onward are reserved for evaluation.

Both the deterministic and generative components operate in normalized data space. Unless stated otherwise, the variables introduced below therefore refer to their normalized representation.

2.3.2 Forecasting setup

ArchesWeather generates forecast trajectories autoregressively. We index forecast lead time by n , which we refer to as *weather time*. At each weather step, the model produces a forecast for the next state x_{n+1} .

Rather than generating x_{n+1} directly, ArchesWeather decomposes the forecast into a deterministic component and a stochastic residual component. The deterministic model first produces a base forecast

$$\hat{x}_{n+1}^{\text{det}}.$$

The corresponding ground-truth residual is defined as

$$r_{n+1} := x_{n+1} - \hat{x}_{n+1}^{\text{det}}. \quad (2.36)$$

A conditional flow-matching model is then trained to generate this residual. At inference time, a sampled residual $\hat{r}_{n+1}$ is combined with the deterministic prediction as

$$\hat{x}_{n+1} = \hat{x}_{n+1}^{\text{det}} + \hat{r}_{n+1}. \quad (2.37)$$

Sampling different residuals while keeping the deterministic forecast fixed therefore produces different members of an ensemble forecast.

2.3.3 Residual flow model

Generating each stochastic residual requires a complete flow-matching trajectory. ArchesWeather therefore involves two nested notions of time: the weather-time index n , which advances the autoregressive forecast, and the flow time t , which describes the generation of a residual within each weather step.

Following the notation introduced in the previous sections, the clean endpoint of the flow is now the forecast residual r_{n+1} , while z_t denotes its noisy state at flow time t .

ArchesWeather parameterizes the Gaussian probability path in terms of a noise level s_t , which decreases linearly from 1 at the beginning of the flow to 0 at its endpoint. The noisy residual is constructed as

$$z_t = (1 - s_t) r_{n+1} + s_t \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I). \quad (2.38)$$

This corresponds directly to the Gaussian probability path introduced in Equation (2.2), with

$$\alpha_t = 1 - s_t, \quad \beta_t = s_t. \quad (2.39)$$

The flow therefore starts from Gaussian noise and progressively approaches the clean residual r_{n+1} .

Unlike the generic flow-matching formulation, in which the neural network directly predicts the velocity field, the ArchesWeather residual model is trained to predict the clean residual from the noisy state. We denote this estimate by

$$\hat{r}_t := r_\theta(z_t, t, c_n), \quad (2.40)$$

where

$$c_n = (x_{n-1}, x_n, \hat{x}_{n+1}^{\text{det}})$$

collects the conditioning information provided to the residual model.

The model is trained using the clean-prediction objective

$$\mathcal{L}_{\text{Arches}}(\theta) = \mathbb{E}_{t, r_{n+1}, \varepsilon} [\|r_\theta(z_t, t, c_n) - r_{n+1}\|_2^2]. \quad (2.41)$$

In the notation of the previous sections, $\hat{r}_t$ thus plays the role of the clean estimate $\hat{x}_t$. Since the flow generates a residual rather than the complete weather state, the corresponding clean weather estimate at flow time t is

$$\hat{x}_t := \hat{x}_{n+1}^{\text{det}} + \hat{r}_t. \quad (2.42)$$

This quantity will later provide the state on which our guidance objectives are evaluated.

2.3.4 Sampling

Although the network predicts the clean residual, sampling requires the corresponding velocity field. From Equation (2.38), the noise associated with the clean estimate $\hat{r}_t$ is

$$\hat{\varepsilon}_t = \frac{z_t - (1 - s_t)\hat{r}_t}{s_t}. \quad (2.43)$$

For the linear path

$$\alpha_t = 1 - s_t, \quad \beta_t = s_t,$$

the velocity is the displacement between the clean residual estimate and the corresponding noise estimate:

$$u_\theta(z_t, t, c_n) = \hat{r}_t - \hat{\varepsilon}_t \quad (2.44)$$

$$= \hat{r}_t - \frac{z_t - (1 - s_t)\hat{r}_t}{s_t} \quad (2.45)$$

$$= \frac{\hat{r}_t - z_t}{s_t}. \quad (2.46)$$

ArchesWeather discretizes the flow into $T = 25$ sampling steps. In the following, $t = 0, \dots, T$ denotes the corresponding discrete flow-step index. Starting from Gaussian noise

$$z_0 \sim \mathcal{N}(0, I),$$

the residual is generated through Euler integration,

$$z_{t+1} = z_t + h_t u_\theta(z_t, t, c_n), \quad h_t = s_t - s_{t+1}, \quad t = 0, \dots, T-1. \quad (2.47)$$

Since the noise level decreases from $s_0 = 1$ to $s_T = 0$, the sequence $\{z_t\}_{t=0}^T$ progressively evolves from noise toward a clean residual.

After the final flow step, the endpoint is taken as the sampled residual,

$$\hat{r}_{n+1} := z_T, \quad (2.48)$$

which is added to the deterministic forecast to obtain

$$\hat{x}_{n+1} = \hat{x}_{n+1}^{\text{det}} + \hat{r}_{n+1} = \hat{x}_{n+1}^{\text{det}} + z_T. \quad (2.49)$$

The resulting nested time structure is illustrated in Figure 3.1, while Figure 2.2 summarizes the computations performed within a single weather step.

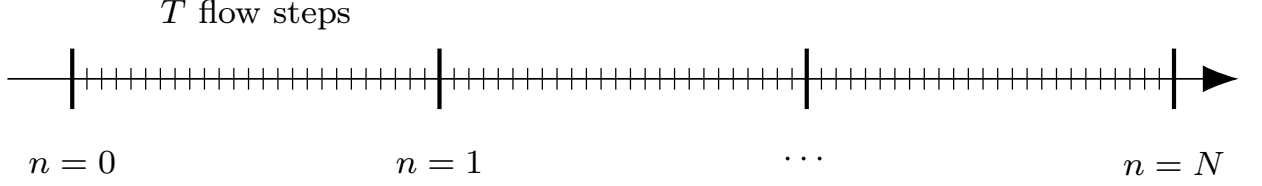


Figure 2.1 – The two nested time scales in ArchesWeather. Each autoregressive weather step $n \rightarrow n+1$ contains a complete flow of $T = 25$ steps used to generate the forecast residual.

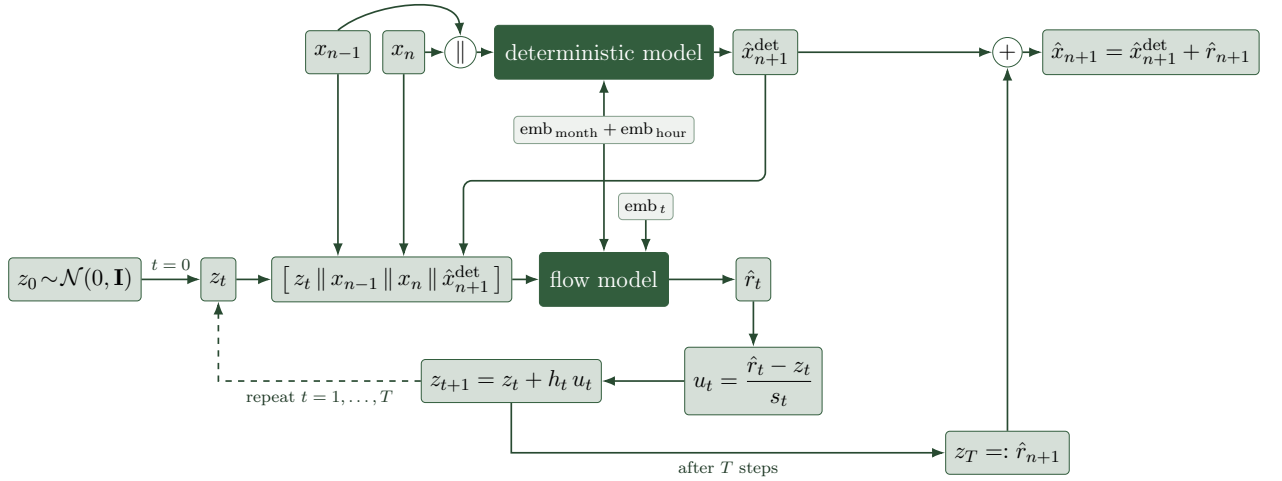


Figure 2.2 – Computational graph of one ArchesWeather forecast step. The deterministic model produces the base forecast $\hat{x}_{n+1}^{\text{det}}$, while the flow model generates the residual through T Euler steps before both components are combined.

This structure will be central to our guidance method, which acts during the residual flow while ultimately steering the resulting weather forecast.

Chapter 3

Background

In this chapter we address the building blocks for the development of our guidance method. We start with a short introduction to flow matching, the generative framework used by ArchesWeather. We proceed by summarizing the idea of guidance and expose a few selected methods. We then outline the relevant implementation details of ArchesWeather, before concluding with a short literature review on the use of guidance in the context of generative weather modeling.

3.1 Flow matching

TODO: somewhere ideas such as the noisy state and others must be introduced here.

TODO: unify expression velocity and vector field

Generative modeling frames the task of generating objects $x \in \mathbb{R}^d$, for instance images or text, as sampling from a data distribution p^1 . In particular, flow matching [4] models the data distribution p_1 as the endpoint of a continuous transport of a tractable prior distribution p_0 , via the probability path $(p_t)_{t \in [0,1]}$. This path is generated by a time-dependent velocity field u_t , defining the ordinary differential equation (ODE)

$$\frac{d}{dt} \phi_t(x) = u_t(\phi_t(x)), \quad \phi_0(x) = x.$$

where ϕ_t denotes the flow (trajectory) of x , whose push-forward carries p_0 to p_t .

A common choice for $(p_t)_{t \in [0,1]}$ is the Gaussian conditional probability path

$$p_t(\cdot \mid x_1) = \mathcal{N}(\alpha_t x_1, \beta_t^2 I) \quad \alpha_t : 0 \rightarrow 1, \quad \beta_t : 1 \rightarrow 0$$

which noise schedulers $\alpha_t, \beta_t \in [0, 1]$ interpolate a data sample with noise, such that

$$\begin{aligned} p_0(\cdot|x) &= \mathcal{N}(\mathbf{0}, \mathbf{I}) \\ p_1(\cdot|x) &= \delta_x. \end{aligned}$$

Samples from the marginal density p_t can then be easily obtained by interpolating $x_1 \sim p_1$ and $x_0 \sim \mathcal{N}(0, I)$ via

$$x_t = \alpha_t x_1 + \beta_t x_0 \sim p_t.$$

To generate samples that follow this probability path, in flow matching we train a neural network u_t^θ set to generate p_t . This is obtained by noting that, although the marginal velocity field is not immediately available to us, it decomposes as an expectation of conditional fields

$$u_t(x) = \mathbb{E}_{x_1 \sim p_1(\cdot|x)}[u_t(x | x_1)]$$

each $u_t(\cdot | x_1)$ generating a simple path from p_0 to a fixed data sample x_1 . (TODO: repetition)
Flow matching exploits the fact that the tractable conditional objective

$$\mathcal{L}(\theta) = \mathbb{E}_{t, x_1 \sim p_1, x_t \sim p_t(\cdot|x_1)} \|u_\theta(x_t, t) - u_t(x_t | x_1)\|^2 \quad (3.1)$$

shares its gradient with the intractable marginal one [4]. Under the linear (optimal-transport) path $x_t = (1-t)x_0 + tx_1$ with $x_0 \sim p_0$, the conditional vector field corresponds to the constant displacement $u_t(x_t | x_1) = x_1 - x_0$, such that training amounts to a least-squares regression onto $x_1 - x_0$.

—somewhere say that since we are drawing from noise we can generate independent samples.
—check notation is aligned with next section

3.2 Guidance

Guidance methods were originally developed in the context of diffusion models trained to generate images. In the following we make a short derivation of the guidance under the perspective of (cite DPS paper) and translate it to fit the flow matching framework. We then discuss the idea of a guidance strength-schedule, and conclude by summarizing other relevant guidance frameworks.

3.2.1 Guidance as posterior sampling

In most cases, we are not interested in sampling an arbitrary object from $p_1(x)$, but rather to condition the generation on a certain property y . Diffusion posterior sampling [...] (TODO: cite) builds on the idea that guided generation can be framed as sampling from the conditional posterior distribution $p_1(x|y)$. The method is derived in short as follows.

Using Bayes rule and taking the gradient with respect to the noisy state we can rewrite the posterior as a sum of scores

$$\begin{aligned} p_t(z_t|y) &\propto p_t(y|z_t) p_t(z_t), \\ \nabla_{z_t} \log p_t(z_t|y) &= \nabla_{z_t} \log p_t(z_t) + \nabla_{z_t} \log p_t(y|z_t). \end{aligned}$$

In general, $p_t(y | z_t)$ is hard to acquire in closed-form due to the dependence on the time t , as there only exists explicit dependence between y and x_1 . We thus rewrite the intractable likelihood in form of marginal distribution, and approximate the two appearing terms inside the integral (also intractable) as follows

$$\begin{aligned} p_t(y|z^t) &= \int p_t(y|x, z_t) p_t(x|z_t) dx \\ &= \int p_t(y|x) p_t(x|z^t) dx \quad y \perp x_t|x \\ &\approx \int p_t(y|\hat{x}_t) p(x|z^t) dx \quad \text{first approximation} \\ &\approx \int p_t(y|\hat{x}_t) \delta_{\hat{x}_t} dx \quad \text{second approximation} \\ &= p_t(y|\hat{x}_t) \end{aligned}$$

where $\hat{x}_t$ is the tractable clean estimate at time t . No conditions are imposed on the approximated likelihood term $p_t(y|\hat{x}_t)$, and can thus be modeled arbitrarily.

3.2.2 From diffusion to flow matching

While the latter expression is immediately applicable to diffusion models, we need to convert the expression first to use it within the flow-matching framework. For Gaussian probability

paths, the unconditional and conditional scores relate to the conditional vector field as follows

$$\begin{aligned} u(z_t | y) &= a_t \nabla_{z_t} \log p_t(z_t | y) + b_t z_t \\ u_t(z_t) &= a_t \nabla_{z_t} \log p_t(z_t) + b_t z_t \end{aligned}$$

Inserting the ?? (TODO: ref the equation!) allows to reduce the conditional vector field to

$$\begin{aligned} u(z_t | y) &= a_t \nabla_{z_t} (\log p(z_t) + \log p(y | z_t)) + b_t z_t \\ &= u(z_t) + a_t \nabla_{z_t} \log p(y | z_t). \end{aligned}$$

The guided vector field can thus be simply expressed as a sum between the unguided vector field and the gradient with respect to the noisy state of the likelihood $p_t(y|z_t)$. Finally, by combining this with the approximation ?? (cite eq), we obtain

$$\begin{aligned} u_t(z_t | y) &= u(z_t) - a_t \nabla_{z_t} \log p(y|z_t) \\ &\approx u(z_t) - a_t \nabla_{z_t} \log p(y|\hat{x}_t) \end{aligned}$$

3.2.3 Guidance strength-schedule

While the latter mathematical framework offers an elegant way to express guidance as sampling from the posterior distribution $p_1(x|y)$, researchers realised that they could obtain better results by scaling up the contribution of the guidance term. Scaling up the guidance term means that guided samples stop being samples from the posterior distribution, trading mathematical sophistication for control.

To unify the notation we can express this idea as follow: replace a_t with a general guidance strength-schedule λ_t , which we can decompose

$$\lambda_t := w \cdot \alpha_t \cdot c_t \tag{3.2}$$

with:

- w : guidance strength (time independent)
- $\alpha_t \in [0, 1]$: guidance profile (not necessarily equivalent to a_t)
- c_t : a general step normalization constant (specific to different guidance algorithms)

TODO: explain why 0,1 -> take it from chapter 4 Note that, setting $w = 1$, $\{\alpha_t\}_{t=1}^T = \{a_t\}_{t=1}^T$, and $c_t = 1$, for $t \in [0, 1]$ retrieves the unconditional vector field of ?? (cite eq.).

Under this notation, we define a modified version of the conditional velocity field

$$\begin{aligned} u_t(z_t \mid y) &= u(z_t) - a_t \nabla_{z_t} \log p(y \mid z_t) \\ &\approx u(z_t) - a_t \nabla_{z_t} \log p(y \mid \hat{x}_t) \\ \tilde{u}_t(z_t \mid y) &:= u(z_t) - \lambda_t \nabla_{z_t} \log p(y \mid \hat{x}_t) \end{aligned}$$

denoting it more simply as *guided velocity field*, and rename its counterpart $u(z_t)$ as *unguided velocity field*.

Different approaches to scale guidance have been proposed. In the following literature review we report upon different approaches, including this point of view.

3.2.4 Guidance methods

Guidance scaling approaches to include:

- scale up w
- change the default profile schedule to an arbitrary α_t
- normalize the step to satisfy a certain constraint, such as setting $\{\lambda_t\}_{t=1}^T$ s.t. the guidance vector represents a fix fraction of the unguided vector field.

In addition to the the choice of different guidance-strength schedules, different approaches model the way the guidance vector is obtained in different ways.

—Since this map from x_0 to x_1 is a single deterministic and differentiable trajectory, its output can be steered by differentiating a target loss through the flow—the mechanism on which our guidance builds (Chapter 5).

FlowGrad

D-Flow

Control paper

FlowGrad [5] backpropagates such a gradient through the sampling trajectory; D-Flow [1] instead optimises the source x_0 , which for Gaussian paths projects the update onto the data manifold; and Wang et al. [8] frame the steering as optimal control. Our method adopts this gradient-based view but, rather than tuning a guidance strength, drives an explicit target gap to zero (Chapter 5).

3.3 ArchesWeather

A wide range of generative models have been successfully applied to weather forecasting. To prototype our method we settle on ArchesWeather [2], chosen for its low computational cost relative to heavier alternatives such as GenCast [6]. This section presents the details relevant to our exposition and defers the rest to the original paper.

3.3.1 Dataset

ArchesWeather is trained on the ERA5 reanalysis dataset [3], in one of the versions preprocessed by WeatherBench 2 [7].

The particular dataset is regridded to a regular 1.5° latitude–longitude grid of 240×121 points. Each weather state is a stack of 82 two-dimensional fields. Four of them are surface fields: 2 m temperature, mean sea-level pressure, and the two components of the 10 m wind. The remaining six are upper-air fields—geopotential, temperature, specific humidity, the two horizontal wind components, and vertical velocity—each resolved at 13 selected pressure levels

$$\{50, 100, 150, 200, 250, 300, 400, 500, 600, 700, 850, 925, 1000\} \text{ hPa},$$

for a total of $6 \times 13 = 78$ level–field combinations.

Weather states are subsampled at 00:00, 06:00, 12:00 and 18:00 UTC. Following the WeatherBench model evaluation protocol, all dates from 01.01.2020 onward are held out as the test set.

3.3.2 Objective

TODO: everywhere remove n and state explicitly that we simplify notation whenever needed. ArchesWeather is a medium range forecaster, predicting up to 15-day trajectories in an autoregressive fashion, and runs two submodels: a deterministic- and a flow-matching-based residual model. The deterministic model predicts a base forecast $\hat{x}_{n+1}^{\text{det}}$, while the residual model is trained to generate the residual

$$r_{n+1} = x_{n+1} - \hat{x}_{n+1}^{\text{det}}$$

where x_{n+1} denotes the ground truth state at the next time step. The two predictions are

then combined to form a probabilistic forecast

$$\hat{x}_{n+1} = \hat{x}_{n+1}^{\text{det}} - \hat{r}_{n+1}$$

This split implies two distinct notions of time (Figure 3.1), which we denote as *weather time* and *flow time*. The weather time ticks on 24 h steps, indexed by $n = 0, \dots, N$, with $n = 0$ corresponding to the current state and $n = N$ the last forecast step. The flow time runs within each weather time step and is indexed by $t = 0, \dots, T$, with $t = 0$ corresponding to pure noise, $t = 1$ to the first sampling step, and $t = T$ to the last sampling step (clean residual).



Figure 3.1 – The two nested time scales: the weather time n advances in 24 h steps, while a full flow of T steps generates each transition.

The flow-matching model samples residuals via Gaussian probability paths with noise schedulers $a_t = 1 - \frac{t}{T}$ and $b_t = \frac{t}{T}$, where $s_t := a_t$ denotes the noise level in the paper’s notation.

Instead of regressing the flow-matching velocity $r_{n+1} - \varepsilon$, the flow model is trained to predict the clean residual $\hat{r}_t := r_\theta(z_t, t, c)$ directly via the loss. Writing $\hat{r}_t := r_\theta(z_t, t, c)$ for the clean residual estimate at flow step t , the training objective is

$$\mathcal{L}(\theta) = \mathbb{E}_{t, r_{n+1}, \varepsilon} \|\hat{r}_t - r_{n+1}\|^2,$$

conditioned on $c = (x_{n-1}, x_n, \hat{x}_{n+1}^{\text{det}})$, and z_t being the noisy residual equivalent to

$$z_t = (1 - s_t) r_{n+1} + s_t \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I).$$

Both models operate directly in normalized data space, so the noise, the residual and the state share the same 240×121 grid across the 82 fields. We defer the details of the two models’ architectures to the original paper.

3.3.3 Sampling

At sampling time the residual prediction is turned back into a flow-matching velocity. Inverting the interpolation for the implied noise $\hat{\varepsilon}_t$ and forming the displacement $\hat{r}_t - \hat{\varepsilon}_t$ gives

$$\begin{aligned}\hat{\varepsilon}_t &= \frac{z_t - (1 - s_t) \hat{r}_t}{s_t}, \\ u_\theta(z_t, t) &= \hat{r}_t - \hat{\varepsilon}_t \\ &= \frac{s_t \hat{r}_t - (z_t - (1 - s_t) \hat{r}_t)}{s_t} \\ &= \frac{\hat{r}_t - z_t}{s_t}.\end{aligned}$$

The residual is sampled by integrating the velocity over $T = 25$ Euler steps

$$z_{t+1} = z_t + h_t u_\theta(z_t, t), \quad h_t = s_t - s_{t+1},$$

with $\hat{r}_{n+1} := z_T$, such that the combined forecast is equivalent to

$$\hat{x}_{n+1} = \hat{x}_{n+1}^{\text{det}} + \hat{r}_{n+1} = \hat{x}_{n+1}^{\text{det}} + z_T.$$

Sampling multiple residuals is then a cheap way to produce ensemble forecasts.

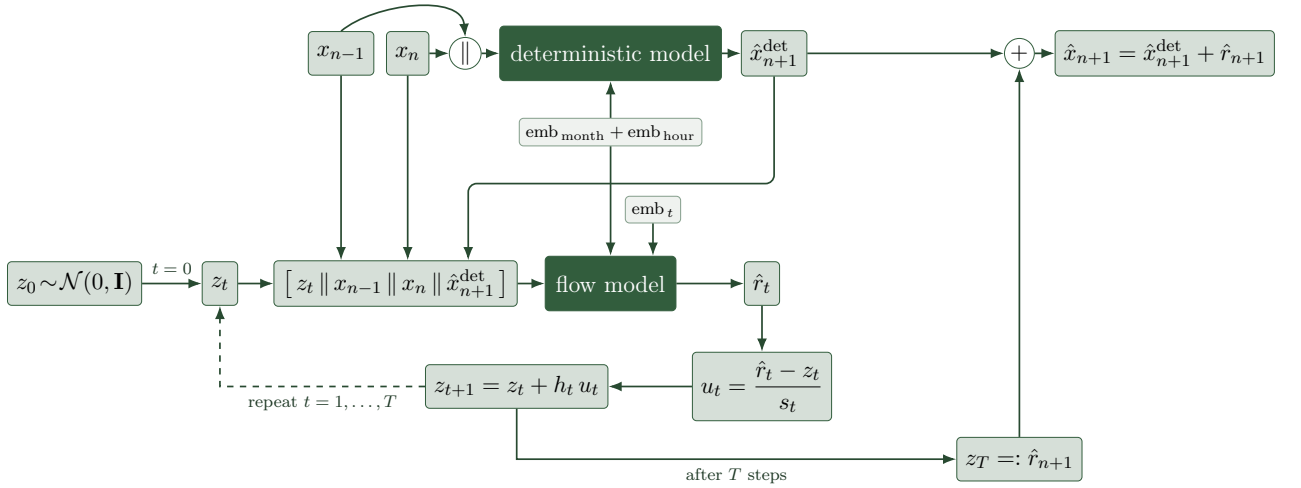


Figure 3.2 – One forecast step of ArchesWeather.

3.4 Guidance in weather modeling

The last piece of the puzzle before presenting our approach is to analyze what people have already tried out in the context of guidance applied on generative weather models, enabling us to expose the relevant gaps that we want to fill.

Digging into the paper archives, I was able to find only three notable papers that were approaching the same general task as us.

The first one is ...

The most similar in nature, and the one from which we will blatantly borrow some aspects of the evaluation, is also the one that approaches guidance and its components the most thoroughly.

We have thus presented all relevant background pieces for the exposition of our contributions. We will spend the next three chapters introducing each of these key aspects.

Chapter 4

Specifying events of interest

In this chapter we expose how we choose to specify events of interest, that is, how we set the guidance target.

At this point it is important to recall that in this thesis we narrow the focus on heatwaves, and will present the methods accordingly. However, the method we develop is variable agnostic (and model agnostic), and could be tuned to generate other types of extreme events, such as hurricanes, floods, etc., following a very similar logic.

As previewed in the introduction, we are interested in guiding the base forecasts by targeting a trajectory of masked averages over the selected variable and region of interest. In our specific use case, we target an increase in the average temperature over a certain region, over N weather steps.

We will spend this chapter explaining how we define the relevant knobs (target parameters) of our heuristic and conclude by examining the induced objects and their relations.

4.1 Target variable and region: mask

We define the region of interest as a simple bounding box around pixels in the 121×240 rectangular spatial grid. A pixel is either included or excluded from the mask.

This approach has three benefits. Firstly, magnitudes of different objects are comparable across mask sizes. Secondly, it enables us to efficiently compute any average of choice inside the mask region. While the mask has non-zero weights only at the target variable field, the mask region can be used to compute arbitrary objects, such as the average of other variables but not only, that have the same longitudinal and latitudinal coordinates as the target variable.

Finally, defining a region with equal weights inside the mask avoids biasing the guidance inside the region, making it clear what area we are targeting in the first place. Take the opposite for understanding: if we defined a Bell-shaped mask, it wouldn't be clear what region we are spefing exactly, and we would be biasing the generation in un predictable manner, making it difficult to predict different across shapes.

Every pixel is weighted in proportion to the area it covers, so that the masked average is a true area average and not biased towards the poles. This is the latitude weighting scheme of WeatherBench 2 [7], also used in the model's training loss.

We index grid pixels by $i \in \{1, \dots, 121\}$ in the latitude direction and $j \in \{1, \dots, 240\}$ in the longitude direction, which give us coordinates (λ_j, φ_i) . We denote the box indicator by k_{ij} (1 inside the region, 0 outside), and cell area by a_i at latitude φ_i . The pixel weights are then

$$m(\lambda_j, \varphi_i) = \frac{k_{ij} a_i}{\sum_{i', j'} k_{i'j'} a_{i'}}, \quad a_i = \int_{\beta_i^-}^{\beta_i^+} \cos \varphi \, d\varphi,$$

where $[\beta_i^-, \beta_i^+]$ is the latitude band of row i , so that a_i is the exact area of the cell. As a consequence cells at the equator now (correctly) carry more weight than ones at the poles.

4.2 Target trajectory

Once we have specified the variable and region of interest, we still have to define the target mask-averaged values, which we want to guide the model towards: a scalar trajectory over N weather steps.

But how can we specify valid target trajectories in the sense of climatologically plausible ones? The answer really depends on the specific use case we have in mind, which would call for different heuristics. We might come up with arbitrary sampling strategy based on historical trajectories that we can query from the dataset at hand.

To simplify matters, in this thesis we define target trajectories as a linear percentage increase from the current state to a prescribed maximum w.r.t. base forecast at a specified step $N - H$. Concretly, we prescribe the target trajectory $\{y_n\}_{n=1}^N$ as follows:

$$y_n = \rho_n \cdot \mathcal{M}(x_n^{\text{ung}}) \tag{4.1}$$

$$\rho_n = \frac{\rho_{N-H}}{N} \quad \text{if } n \leq N - H \text{ else } 0 \tag{4.2}$$

Defining the trajectory using the percentage approach enables us to analyze different levels of

intensities across different settings more easily. What we are in fact doing though, is defining the target temperatures.

Explicitly defining a target trajectory allows us to separate clearly the target of the guidance, and the search for optimal parameters of the guidance method. While the papers summarized in chapter 2 all play around with different guidance strengths to achieve different results, we take an orthogonal approach: we prescribe different guidance objectives, and then retrieve the most adequate parameters that bring us there in a separate step. These two definitions are really equivalent in terms of the retrieved parameters and the produced outcome under the same guidance framework, however, the latter approach has following benefit: if we can define desired outcomes a priori, we do not have to tune parameters and discard experiments that do not achieve desired results, we can optimize them directly.

This simple realization is the main contribution of our work. The rest of the contributions are developed as a consequence.

4.3 Pipeline overview

We have said that we guide with respect to the base forecast, that we define a variable and a region of interest, and that we prescribe the target trajectory for the masked-average that we want to guide towards. In practical terms this means that we will specify the start state we are interested in, produce the base forecast rollout over prescribed N steps, and then define our target trajectory (or multiple ones) for the guidance. We can visualize this scheme as follows.

Chapter 5

Guidance design

We start the chapter with the derivation of the general outline of the algorithm we will use for guidance. We then proceed in exposing different ways to specify its two main components: the loss and the strength-schedule. Importantly, the specific choice of loss and strength-schedule defines the concrete algorithm variants that we derive from the general method. Like the class-object relationship in programming, where the object is a specific instance of the abstract class.

5.1 Base guidance method

5.1.1 Adapting DPS

One popular approach is to model it with a loss function as $p_t(y|\hat{x}_t) \propto \exp(-\mathcal{L}(\hat{x}_t, y))$ such that

$$\begin{aligned} u_t(z_t|y) &\approx u_t(z_t) + a_t \nabla_{z_t} \log p_t(y|\hat{x}_t) \\ &= u_t(z_t) + a_t \nabla_{z_t} \log \exp(-\mathcal{L}(\hat{x}_t, y)) \\ &= u_t(z_t) - a_t \nabla_{z_t} \mathcal{L}(\hat{x}_t, y) \end{aligned}$$

which is also the approach we will take, see chapter 4.

—Since this map from x_0 to x_1 is a single deterministic and differentiable trajectory, its output can be steered by differentiating a target loss through the flow—the mechanism on which our guidance builds (Chapter 5). —A key finding is that in order to make guidance actually work, we have to abandon probability land, by increasing the strength of the guidance. In fact, when

we start to play with λ , the guided output is not a sample of the conditional probability distribution anymore.

In our context we want to generate weather states and guide them towards an extreme event of our choice (in our case increasing temperature). Using the definition of classifier guidance provided in chapter 2, we can design the loss function to achieve this objective.

With y now being the guidance target, we can define a loss $\mathcal{L}(\hat{x}_t(z_t), y)$, dependent on the clean state estimate $\hat{x}_t$, which is a function of the noisy state z_t . With this formulation, we can compute the loss at any step of the flow on $\hat{x}_t$ with $0 < t < T$ and on the final sample $\hat{x}_T = x_T$.

This enables us to define the likelihood term. This is however still not enough to compute the posterior, since it's an intractable integral over all possible clean states

$$p(y | z^t) = \int p(y | x) p(x | z^t) dx$$

We approach the problem like in the DPS paper and define the likelihood term as $p(y | x) \propto e^{-\mathcal{L}(x)}$ with $x := x_T$. This and approximate the intractable expectation by the clean-state estimate at the current flow step t ,

$$p(y | z^t) = \int p(y | x) p(x | z^t) dx = \mathbb{E}_{x|z^t} [e^{-\mathcal{L}(x)}] \approx e^{-\mathcal{L}(\hat{x}_t)}.$$

(this we do like in the DPS paper)

Substituting this approximation in the bayesian formula we get:

$$\begin{aligned} \nabla_{z^t} \log p(z^t | y) &= \nabla_{z^t} \log p(z^t) + \nabla_{z^t} \log e^{-\mathcal{L}(\hat{x}_t)} \\ &= \nabla_{z^t} \log p(z^t) - \nabla_{z^t} \mathcal{L}(\hat{x}_t) \end{aligned}$$

In the context of flow matching, this general formulation translates to:

$$\begin{aligned} \tilde{u}(z_t) &= u(z_t) - a_t \nabla_{z_t} \log p(y|z_t) \\ &= u(z_t) - a_t \nabla_{z_t} \mathcal{L}(\hat{x}_t) \end{aligned}$$

where $a_t = \dots$ in the specific context of our model.

This means that to sample from the approximate posterior distribution we simply have to compute the right term at the specific step in the flow.

5.1.2 Guidance strength-schedule

Under the adapted DPS framework we just introduced, we are now free to define any guidance schedule of our choice. To do so, we decide to test different schedules empirically.

We note that, under the define decomposition of λ_t , the role of α_t is only relative to the flow steps. It can thus be rescaled between 0 and 1 via a min-max scaling. We defer the rest of the discussion to chapter 6.

5.1.3 Base algorithm

Putting it all together we have described following base algorithm:

1. Estimate the clean state from the current noisy residual: $\hat{x}_t = \hat{x}^{\text{det}} + \hat{r}_t$.
2. Score it with the masked loss: $\mathcal{L}(\hat{x}_t)$.
3. Take the gradient with respect to the noisy state: $g_t = \nabla_{z^t} \mathcal{L}(\hat{x}_t)$.
4. Modify the velocity: $\tilde{u}_t = u_t - \lambda_t g_t$ with $\lambda_t = w \cdot a_t \cdot c_t$.
5. Take the Euler step as usual: $z^{t+1} = z^t + h_t \tilde{u}_t$.

Two design choices define the method:

- the **guidance schedule** — how hard and when.
- the **masked loss** — what to push and where;

Our implicit assumption here is that the gradient encodes the most likely shift to be applied under the learned data distribution to achieve the set target. This is only an hypothesis however and we do not test it. The reasoning we follow is the one adapted by the classifier guidance framework.

Note that, while classifier guidance based on a loss function is a viable option, researcher in the literature use the same base algorithm, but optimize the guidance term as adversarial noise instead of using the gradient. In our preliminary experiments we note that computing an adversarial perturbation to guide the generative model produces unrealistic results, compared to the gradient based perturbations. For this reason we use the adversarial perturbations as baseline.

We have thus defined previously the mask and the target profile in Chapter 4. In the next we will address how these are used by the masked loss. We then address the question regarding the guidance schedule in the subsections exposing the three developed guidance algorithms.

5.2 Guidance design choices

5.2.1 Guidance schedule

— present here the relation between the three parts of *lambda* and the fact that the α_t curve can always be rescaled between 0 and 1, s.t. different curves are the same (the relative strength between the steps is what counts here).

5.2.2 Guidance loss

One of the main contributions of this thesis is the realization that in the weather domain, we can define a guidance target, which can be achieved precisely. By precisely I mean that the loss can be driven to zero. While in the image domain, there is always a ground truth during training, at inference we play with different guidance strengths to identify a range where the generated images are good looking. In the weather domain however, we can define a target that can be closed by tuning the guidance strength.

This is great because instead of tuning the guidance strength to find a range of plausible results, we can define what plausible results look like a priori (guidance target sampling trajectory defined through historical data and plausibility tests), and optimize the guidance strength to achieve this target.

Remember from section 3 that we decided to prescribe the guidance target value at each weather step.

We define the guidance loss as follows:

$$\mathcal{L} = \left(\underbrace{\mathcal{M}(\hat{x}_t)}_{\text{current}} - y \right)^2 = \xi_t^2$$

$$\mathcal{M}(x) = \sum m \odot x, \quad \text{masked average with } \sum m = 1$$

where $\hat{x}_t$ is the clean-state estimate at flow step t and e_t is the gap (signed difference).

We realize that since we are pushing the base forecast (gui-ung object), this is equivalent to

state the following:

$$\mathcal{L} = \left(\underbrace{\mathcal{M}(\hat{x}_t)}_{\text{current}} - \underbrace{(1 + \phi) \mathcal{M}(x^{\text{gui_ung}})}_{\text{target}} \right)^2$$

$$\phi = \frac{\mathcal{M}(x^{\text{gui_ung}}) - y}{\mathcal{M}(x^{\text{gui_ung}})}$$

where ϕ_n is the intervention profile (the imposed change) in percentage. The reason to do this is that it provides a practical way to visualize guided results against the base forecast from different perspectives, which also include ϕ . Another one is that this enables us to test our methods against intensity levels in terms of simple percentages, without the need to go through the pipeline defined in the previous chapter.

In addition to the gap ξ , we could to the loss arbitrary regularization terms encoding different desiderata. Writing it down explicitly, the loss would then take the general form:

$$\mathcal{L} = \xi^2 + \gamma \mathcal{R}$$

Note that the gradient flows through the model into the full noisy residual z_t , because the network Jacobian is defined over the whole state.

We essentially have a very similar setting to the guided diffusion for precipitation paper, but we make it more general in these ways:

- we define a guidance target explicitly, in contrast to observe results across different lambdas
- we experiment with different a_t trajectories, in addition to guiding computing the gradient at the first step
- we test level of intensity by shifting the intensity parameter from the flow (lambda) to the trajectory definition (ϕ_n)
- Our loss is driven to zero in terms of the difference from the target, instead of being minimized iteratively (a more explicit way to encode the target scenario)

5.3 Specific guidance methods (instances)

In this section we develop different variants of classifier free guidance, which we adapt to this setting. We explore the different variants, tune them, expose their flaws, assess their computational requirements, and compare their respective trade off. In the end we close in on

the most promising method in terms of reliability and computational requirements, evaluated both in terms of theoretical cost, but also in terms of practical factors such as difficulty of tuning and potential need for multiple runs.

5.3.1 CLOSE-GAP

Given the loss defined above, we can define a guidance method that simply closes the gap to the target as prescribed by the guidance schedule in a greedy fashion.

At each step, the algorithm performs following tasks:

- Compute the gap and the gradient
- Compute the guidance factor λ_t s.t. $m(\text{step} + 1) - m(\text{step}) = (1 - \eta) \cdot e$ (by normalizing the gradient)

At the end of the flow we will have closed the gap up to $(1 - \eta)^{T-1}$.

As we will present in the experiments section, the method tends to produce a high oscillatory behavior, because the unguided steps will overshoot the scheduled gap if we push too much. This happens whether we want to close the theoretical gap or the actual gap.

If we knew a priori the optimal guidance strength to apply at each step, the CLOSE-GAP method would be optimal in terms of convergence and stability. However, we do not know the optimal guidance strength, and therefore we have to tune it. This means that we trade off the number of optimization steps we take to find the optimal guidance strength, with the number of experiments to tune for the optimal guidance strength schedule.

The method thus trades its efficient greediness, with the need to tune η , before obtaining a more well behaved guided trajectory, thus requiring multiple runs, making it much more costlier in its practical application.

5.3.2 OPTIMIZE-SCHEDULE

We attempt to overcome the issue of the fix step sizes by optimizing for λ_t directly. To do so we add a regularization term to the loss to encode a well-behaviourness criteria of our choice. We could dream up arbitrary penalty terms such as angular deflection, For simplicity we choose to penalize the total guidance needed to reach the set target. Ideally this would not generate the oscillatory behaviour, since then we would need to guide downwards in that case. We add the regularization term to the loss as follows: ...

We then optimize λ_t by using a ... method.

Note that this time the step is allowed to take the gradient norm into consideration.

In theory this method promises to find us the schedule that minimizes the total guidance needed to obtain the guidance target. However in practice this optimization is really hard to perform, because of the high dependance of λ^* on its initialization. The problem is related to the fact that the gradient applied for the guidance at each step changes with respect to the schedule. So the optimizer will converge very fast to a local minimum, highly influenced by the initialization of the schedule.

5.3.3 OPTIMIZE-GAIN

As final method we develop one that solves the oscillatory behaviour of the first method, and removes the high dependency on the initialization of the second method, by encoding it explicitly, thus requiring lower computational requirements.

The approach of this method is to fix the a_t schedule (in contrast to the second method), but allowing the gradient magnitude to play a role (in contrast to the first method), and optimize for a w that achieves the set target over the T flow steps.

We realize that the dependence on w of the loss is almost linear (TODO: check empirically), so we can use a simple secant method that converges reliably and fast.

More concretely, the algorithm works as follows:

- Initialize w to a lower value to one that we know works empirically
- Make the flow with that w
- Compute the loss and compute two secant points
- Set the middle point as the new w

Across the methods this is the one that is easier to tune and works most reliably across settings. We will see this empirically in section 6.

5.3.4 Comparison

TABLE

- approach
- hyperparameters
- time complexity

- memory requirements

Chapter 6

Evaluation

We evaluate our guidance method against the two requirements we set for it: *achievement*, the ability to reach the specified target, and *realism*, the physical plausibility of the states it generates.

The first requirement concerns whether the algorithm reaches the target, and how well it does so across settings. We test this through convergence, stability, and trajectory deviation, which together expose the failure modes of the algorithm and reveal which design choices are more adequate overall.

The second requirement concerns whether the generated states are realistic, in terms of temporal, spatial, vertical, and cross-variable coherence.

The evaluation is organised as a funnel. In *flow-time* we assess the sampling dynamics of the algorithm variants and guidance schedules, and select the most promising method. In *weather-time* we then characterise the parameters of the experimental setting, select the settings that produce the most desirable results, and finally validate the realism of the generated trajectories. Chapter 7 carries out each stage in turn and reports the concrete settings and results; this chapter defines only *what* we test and *how* we score it.

Alongside these tests, the discussion examines further aspects that the framework does not measure directly—such as the diversity of the generated trajectories and the informativeness of the gradient—but that bear on the overall value of the method and its applications.

6.1 Flow-time assessment

Here we assess the algorithm variants and the guidance schedules, with the purpose of selecting the most promising method and parameters for the next set of experiments. Before asking whether the method produces realistic results, we first have to establish that it works: that it reaches the target, does so reliably across settings, and does so in a stable manner.

6.1.1 Tests

We test the achievement requirement and well-behavedness of the trajectory for different guidance algorithms through the following tests:

T1 – Convergence

The gap to the target is driven to zero. We report the mean and standard deviation of the final gap.

T2 – Stability

The trajectory is free of oscillatory behaviour. We measure this simply with respect to the pushback measured in the average trajectory (gui vs. ung steps).

T3 – Trajectory deviation

The guided trajectory does not deviate excessively from the unguided one. We report the distance between the two in PCA latent space, and the cosine similarity between the steps.

T4 – Tuning complexity

The algorithm parameters can be tuned reliably such that it combines its theoretical desirable properties to its practical applicability (TODO: should be the name of the test).

It's not clear a priori whether total guidance needed to achieve the target is also something to aim for. For completeness we add this in the stability test reports.

We use these tests as criteria to select the most promising method and its an adequate set of parameters for the tests at weather time.

6.2 Weather-time assessment

In weather-time we first characterise the parameters of the experimental setting—to describe their properties and to select settings for the larger experiments—and then validate the realism of the resulting trajectories. Throughout, we pay special attention to the diversity of the produced results as a secondary property of interest, alongside realism.

6.2.1 Parameter characterisation tests

In the following we provide a set of qualitative tests to better understand the role of the main parameters of the guidance at weather time. We test parameters in isolation, using the remaining parameters as controls. We proceed in three stages, examining in turn the region of interest, the variable of interest, and the intervention profile. For each we show different charts and metrics to expose what happens across these settings.

— maybe here is where the baseline would shine.

The reason for these tests is mostly to enhance our intuition about the role of each parameter, and provide a guideline on to select parameters for the large use cases we aim to produce with this work.

Region of interest

Here we assess variability of results by changing size, shift (from center), and coast-land covering of region of the interest.

Variable of interest

For simplicity we focus on heatwaves, and therefore on surface temperature. To probe the informativeness of the gradient, in this test we push the most correlated variables in an attempt to reproduce the pattern obtained by pushing temperature directly.

Intervention profile

Here we assess the degradation of the target patterns across intensity profiles. The intuition is that at some point things should brake down in some trivial way.

6.2.2 Realism tests

Once the guidance method and its parameters are selected, we assess the realism of the generated trajectories. Realism is central to this work, because we deliberately guide the generative model, possibly outside its training distribution. The tests themselves, however, are not specific to this work: they concern the general problem of evaluating the coherence of data-driven models, namely that the generated states should be consistent with the ground-truth dataset in both the guided and unguided settings.

We assess coherence to the ground-truth dataset along four dimensions:

T4 (Temporal).

Transitions of a variable across weather steps should be smooth.

T5 (Spatial).

Patterns over the region of interest should be spatially coherent.

T6 (Vertical).

Patterns should be coherent across pressure levels. We test this using a level rank coherence metric.

T7 (Cross-variable).

Patterns should be coherent with the learned weather transition function across variables. However, this is difficult to test quantitatively, we thus only make a qualitative assessment. (TODO: use the expert knowledge here)

We benchmark these properties against the ground-truth data as anchor (reference set), and against the unguided forecast as baseline, comparing unguided, guided-minus-unguided, and guided states. For T4–T6 we compute difference maps along the axis of interest and report the min, mean, max, and standard deviation across maps. For T7 we compute the cross-variable distributions over the weather steps of the ground-truth data, and report the deviation of the unguided, guided-minus-unguided, and guided states from their average.

For all four tests we average over the experiments and report the mean and standard deviation across runs. This indicates whether the generated extremes are faithful to the data distribution. A subtle but important point: for this to be meaningful, the base forecast must have missed the potential heat spots. The extreme is not only about the average—which the forecast most likely covers faithfully—but about the extreme spots, so both the mean and the max must be tested.

Chapter 7 sets out the concrete settings for each stage of this framework and reports the results, which Chapter 8 then discusses.

6.3 Method Validation

This approach can be used to guide any state towards an extreme over the specified mask.

To validate that the method is doing something sensible we can retrieve real heatwaves from the test dataset (everything that is not included in the train set), and observe what the model is doing.

Chapter 7

Experiments

This chapter carries out the evaluation framework of Chapter 6. Each section follows the same pattern: we first state the concrete settings, then report the results and the decision they lead to. Tests are referenced by the identifiers introduced in Chapter 6 (T1–T7).

For all experiments we select the region of interest and retrieve the state corresponding at the start of the heatwave reaching the highest temperatures over the mask with selection criteria being the highest rolling window sum over 5 days.

7.1 Flow-time evaluation

7.1.1 Setup

Here we assess the performance of the three proposed guidance algorithms with following setup:

- 3 different starting states with different masks (Europe, America, Australia)
- $N=2$, $M=3$
- 2 intensity levels (+0.25%, +0.75%)
- 2 a_t schedules (spike, closegap($\eta = 1$))

7.1.2 Achievement tests

The difficulty of tuning the first and second algorithm make them less suitable than the third one. Regarding algorithm 1, having a lower computational complexity is useless in practice if we then have to perform multiple costly runs. Regarding algorithm 2, having a desirability

requirement encoded in the loss itself only complicate things. Tuning the optimization of this algorithms outweighs the benefits, and induces a much higher cost in practical terms to the algorithm.

At later stages, the gradient becomes more similar to the mask (is more diluted, less sparse), which makes the guided pattern at the end of the flow be more spread.

Using a spike@1 approach is essentially equivalent to shifting the starting point of the flow trajectory (especially for the region over the mask) and let particle flow across the learned ODE.

What we observe is that the model doesn't necessarily pushes back against us. In contrary, when the method is tuned well, the model will do most of the pushing itself (look at this in percentage).

7.2 Weather-time evaluation

Throughout the weather-time experiments we vary the start state while fixing $N = 2$ and $M = 3$: this controls for the effects of $N = 1$ and for seed-specific effects, while keeping computational and memory requirements as low as possible.

Note that we will use the same experiments for the parameter characterization tests and for the realism tests.

— the realism tests could simply be used in each parameter subsection

7.2.1 Parameter characterisation tests

Region of interest

Variable of interest

Intervention profile

7.2.2 Realism tests

7.3 Use cases

For the final use cases we fix the parameters selected above and increase N and M , producing the experiments we had in mind when setting the goals of the project.

7.3.1 Europe 2020-06-01

7.3.2 Australia 2020-01-01

7.3.3 North America 2020-07-01

Chapter 8

Discussion

8.1 Mask

We experimented with a gaussian mask, but realized in our experiments that the best thing to do is equal weighting inside the region. The correct way to define the region of interest is to choose a greater mask size instead of using some weighting scheme fading out of an epicenter. The mask size should be chosen to cover the entire area of interest, ensuring that all relevant data is included in the analysis. A larger mask size can help to reduce noise and improve the accuracy of the results, while a smaller mask size may exclude important information. It is important to carefully consider the appropriate mask size for each specific application to achieve optimal results. Either way we (most probably) loose the global coherence for the guided weather states, so we might as well define directly the region of interest with a mask. Additionally, this enables us to actually make statements about "the region of interest", having equal weights everywhere. Introducing different weights might bias the generation in a way that is not desired.

We will see that including or not a specific area will change the guided patterns substantially. This however is consistent with the notion of "region of interest". If we consider a greater area for the changes, the changes will be applied respectively.

— note here: it may also be that the pushed area is coherent with the rest, and that the next deterministic prediction influences other regions too. It is really important to understand how to evaluate what we are doing here. Since we are aiming at a specific region of interest, and we are guiding mainly locally over the region of interest, losing the global coherence of the weather states. We shall thus include this in our evaluation by comparing inside region, outside region, and full spatial domain.

8.2 ...

The algorithm doesn't produce diverse results, because the gradients are really stable. This has to do with the fact that we have learned one and only one flow model (transition function). If we had multiple residual diffusion model, we would expect the possible effects to be more diverse. However, the second interpretation might be that the gradient provides with the information about the most likely direction of change (which is true under the learned data distribution).

This happens because the guidance vector is a scaled version of the flow model's gradient wrt to the noisy state. The gradient is stable across the full trajectory, because the only variable inputs are the noise itself and the timestep of the flow. The stochasticity of the noise is not enough to produce diverse results. The direction of the gradient is always the same. If we say "let the masked average go up by ϕ percent", the only thing that changes is the magnitude of the gradient, but not its direction, which is specified by the Jacobian.

This is in contrast to guidance settings.

Shift between atmosphere and stratosphere.

Put here all observations

Chapter 9

Conclusion

9.1 Summary

9.2 Future Directions

9.3 Conclusive Remarks

Appendices

Appendix A

Code-base structure

Bibliography

- [1] Heli Ben-Hamu et al. “D-Flow: Differentiating through Flows for Controlled Generation”. In: *International Conference on Machine Learning (ICML)*. 2024.
- [2] Guillaume Couairon et al. “ArchesWeather & ArchesWeatherGen: A Deterministic and Generative Model for Efficient ML Weather Forecasting”. In: *arXiv preprint arXiv:2412.12971* (2024).
- [3] Hans Hersbach et al. “The ERA5 global reanalysis”. In: *Quarterly Journal of the Royal Meteorological Society* 146.730 (2020), pp. 1999–2049. DOI: 10.1002/qj.3803.
- [4] Yaron Lipman et al. “Flow Matching for Generative Modeling”. In: *International Conference on Learning Representations (ICLR)*. 2023.
- [5] Xingchao Liu et al. “FlowGrad: Controlling the Output of Generative ODEs with Gradients”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023.
- [6] Ilan Price et al. “GenCast: Diffusion-Based Ensemble Forecasting for Medium-Range Weather”. In: *arXiv preprint arXiv:2312.15796* (2023).
- [7] Stephan Rasp et al. “WeatherBench 2: A Benchmark for the Next Generation of Data-Driven Global Weather Models”. In: *Journal of Advances in Modeling Earth Systems* 16.6 (2024), e2023MS004019. DOI: 10.1029/2023MS004019.
- [8] Luran Wang et al. “Training-Free Guided Flow-Matching with Optimal Control”. In: *International Conference on Learning Representations (ICLR)*. 2025.

Eidesstattliche Erklärung

Der/Die Verfasser/in erklärt an Eides statt, dass er/sie die vorliegende Arbeit selbständig, ohne fremde Hilfe und ohne Benutzung anderer als die angegebenen Hilfsmittel angefertigt hat. Die aus fremden Quellen (einschliesslich elektronischer Quellen) direkt oder indirekt übernommenen Gedanken sind ausnahmslos als solche kenntlich gemacht. Die Arbeit ist in gleicher oder ähnlicher Form oder auszugsweise im Rahmen einer anderen Prüfung noch nicht vorgelegt worden.

.....
Ort, Datum

.....
Unterschrift des/der Verfassers/in